

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA PRÁCTICA 3 - PRAC 3

Presentación

En la Práctica 3 se han implementado dos microservicios del caso **CultureInDaHouse**. En esta Práctica 3 probaremos (testing) uno de los dos microservicios y validaremos que nuestro código sigue una arquitectura hexagonal.

La Práctica 3 abarca los contenidos de los libros "Microservices Patterns" y "Fundamentals of Software Architecture" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta Práctica se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la Práctica 3 son:

- Reconocer los factores de testing y calidad de una especificación/diseño.
- Evaluar formalmente el testing y la calidad de una implementación.
- Identificar los modelos más importantes y estándares de calidad de software.

Descripción de la Práctica 3 a realizar

La Práctica 3 se compone de 3 ejercicios. En cada uno de ellos hay que describir **cómo se ha hecho el ejercicio** y aportar **instrucciones** para su correcta ejecución.

Ejercicio 1 (5 puntos)

Partiendo de la solución de la PRAC2 publicada (<https://github.com/ppinedar/epcsd-spring-showcatalog-solucio>), tenéis que desarrollar **pruebas unitarias**, usando **JUnit**, **Spring Boot Test**, **Mockito** y **AssertJ**.

Notas: Podéis usar como referencia para ayudaros a desarrollar los tests:

- <https://github.com/anirban99/hexagonal-architecture/tree/master/src/test/java/com/example/hexagonal/architecture>
- <https://github.com/eugenp/tutorials/blob/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/testing/EmployeeServiceImplIntegrationTest.java>
- <https://spring.io/guides/gs/testing-web/>
- <https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing.spring-boot-applications.with-mock-environment>

Consejo: En las clases unitarias no hacemos uso de ninguna infraestructura ni dependencia real. No usamos comunicación HTTP, servidores, bases de datos, servicios externos u otros microservicios. Todo aspecto relacionado con dependencias se tiene que simular (con **Mockito**), excepto las clases de dominio (Show, ...).

QUÉ HAY QUE HACER

Desarrollar las siguientes clases y tests:

1. **ShowUnitTest.** Incluir un test sobre la clase de dominio *Show* que verifique que al crear un *Show* y cancelarlo, su *Status* final es CANCELLED.
2. **CatalogServiceUnitTest.** Incluir tests que verifiquen:
 - Método Test 1: Cuando llamamos al método *findShowById* con un *id* existente/válido, obtenemos correctamente un show.
 - Método Test 2: Cuando llamamos al método *findShowById* con un *id* inexistente, obtenemos la excepción que indica que no se ha encontrado.

Nota: La dependencia con el *CatalogRepository* tiene que ser simulada.

3. **CatalogControllerUnitTest.** Incluir una Test que verifique que si llamamos a */categories* obtenemos correctamente las categorías.

Notas: No se puede levantar el servidor para hacer la prueba. Se tiene que simular la infraestructura web para recibir y tratar la petición REST, así como la dependencia con *CatalogService*.

Solución

ShowUnitTest

```
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

public class ShowTest {

    @Test
    void should_change_status_when_canceled() {
        Show actualShow = Show.builder().name("Kiss - The final tour").build();
        actualShow.cancel();
        assertThat(actualShow.getStatus()).isEqualTo(Status.CANCELLED);
    }
}
```

CatalogServiceUnitTest

```
@ExtendWith(SpringExtension.class)
@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class CatalogServiceUnitTest {

    @MockBean
    private CategoryRepository categoryRepository;

    @MockBean
```

```
private ShowRepository showRepository;

@Bean
private KafkaTemplate<String, Show> kafkaTemplate;

@Autowired
private CatalogService catalogService;

@Test
void should_find_show_by_id() {
    Category categoryMusicShow = Category.builder().name(MUSIC_SHOWS).build();

    Show actualShow = Show.builder().name("Kiss - The final
tour").category(categoryMusicShow).capacity(45000L).build();

    Long actualShowId = 120L;

    Mockito.when(showRepository.findShowById(actualShowId)).thenReturn(Optional.of(actualShow));

    Show show = catalogService.findShowById(actualShowId).get();
    assertThat(show)
        .usingRecursiveComparison()
        .isEqualTo(actualShow);
}

@Test
void should_not_find_show_by_id() {
    Long notRealId = 120L;

    Mockito.when(showRepository.findShowById(notRealId)).thenReturn(null);

    Optional<Show> show = catalogService.findShowById(notRealId);

    assertThat(show).isNull();
}
}
```

CatalogControllerUnitTest

Prerequisito: Application para Testing:

```
@SpringBootApplication
public class ShowCatalogApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShowCatalogApplication.class, args);
    }
}
```

Clase:

```
@ExtendWith(SpringExtension.class)
@WebMvcTest(value=CatalogRestController.class)
class CatalogControllerUnitTest {

    private static final String MUSIC_SHOWS = "Music shows";
    private static final String BASKET_SHOWS = "Basket shows";
    private static final String REST_CATEGORIES_PATH = "/categories";

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private CatalogService catalogService;

    @Test
    void find_categories_should_return_all() throws Exception {
        Category categoryOne = Category.builder().name(MUSIC_SHOWS).build();
        Category categoryTwo = Category.builder().name(BASKET_SHOWS).build();

        List<Category> categories = Arrays.asList(categoryOne, categoryTwo);
        Mockito.when(catalogService.findAllCategories()).thenReturn(categories);
    }
}
```

```
mockMvc.perform(get(REST_CATEGORIES_PATH).contentType(MediaType.APPLICATION_JSON))
    .andDo(print())
    .andExpect(status().isOk())
    .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
    .andExpect(jsonPath("$", hasSize(2)))
    .andExpect(jsonPath("$[0].name", is(MUSIC_SHOWS)))
    .andExpect(jsonPath("$[1].name", is(BASKET_SHOWS)));
}
```

Ejercicio 2 (3 puntos)

Partiendo de la solución de la PRAC2 publicada (<https://github.com/ppinedar/epcsd-spring-showcatalog-solucio>), tenéis que desarrollar **pruebas de integración**, usando **JUnit**, **Spring Boot Test** y **AssertJ**.

Notas: Podéis usar como referencia para ayudaros a desarrollar los tests:

- <https://github.com/eugenp/tutorials/tree/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/testing>

Consejo: Las pruebas de integración tienen que probar que el servicio se comunica con sus dependencias reales y la infraestructura. Por lo tanto, no se hace uso de la librería Mockito como en las pruebas unitarias.

QUÉ HAY QUE HACER

Desarrollar la siguiente clase con su test:

CatalogRepositoryIntegrationTest. Probar que al almacenar una categoría (usar *entityManager*) podemos encontrarla correctamente con el método *findCategoryById* de la clase *CategoryRepository*.

Solución

CatalogRepositoryIntegrationTest

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@ContextConfiguration(classes = TestConfig.class)
class CategoryRepositoryIntegrationTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private CategoryRepository categoryRepository;

    @Test
    void whenFindByName_thenReturnCategory() {
        String categoryName = "Music Shows";
        Category category = Category.builder().name(categoryName).build();
        CategoryEntity categoryEntity = CategoryEntity.fromDomain(category);

        entityManager.persistAndFlush(categoryEntity);

        Category fromDb = categoryRepository.findCategoryById(categoryEntity.getId()).get();
        assertThat(fromDb.getName()).isEqualTo(category.getName());
    }
}
```

Ejercicio 3 (2 puntos)

Partiendo de la solución de la PRAC2 publicada (<https://github.com/ppinedar/epcsd-spring-showcatalog-solucio>), queremos verificar que la arquitectura se basa correctamente en una arquitectura hexagonal y que usamos buenas reglas arquitectónicas.

Notas: Para hacer el test usaremos **ArchUnit** (<https://www.archunit.org/>) con **Junit5**. Usad la versión 0.23.0 de ArchUnit.

Lecturas recomendadas:

- Capítulo “Measuring and Governing Architecture Characteristics” del libro “Fundamentals of Software Architecture”.
- Documentación ArchUnit:
https://www.archunit.org/userguide/html/000_index.html#_junit_4_5_support
- Ejemplos ArchUnit con JUnit5: <https://github.com/tng/archunit-examples/tree/main/example-junit5/src/test>

QUÉ HAY QUE HACER

Desarrollar los siguientes 2 tests automáticos de verificación de la arquitectura sobre el microservicio *ShowCatalog*:

- Se cumple la arquitectura hexagonal (también llamada *onion*)
- Las clases que se encuentran en el *package domain.service* y están anotadas con `@Service`, tienen su nombre acabado en **ServiceImpl**.

Solución

```
import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;
import org.springframework.stereotype.Service;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.*;
import static com.tngtech.archunit.library.Architectures.onionArchitecture;
```

```
@AnalyzeClasses(packages = "edu.uoc.epcsd.showcatalog", importOptions = { ImportOption.DoNotIncludeTests.class,
ImportOption.DoNotIncludeJars.class })

public class ArchitectureTest {

    @ArchTest
    static final ArchRule domain_service_with_spring_annotation =
        classes()
            .that().resideInAPackage("..domain.service..")
            .and().areAnnotatedWith(Service.class)
            .should().haveSimpleNameEndingWith("ServiceImpl");

    @ArchTest
    static final ArchRule onion_architecture_is_respected = onionArchitecture()
        .domainModels("..domain..")
        .domainServices("..domain.service..")
        .applicationServices("..application..")
        .adapter("persistence", "..infrastructure.repository..")
        .adapter("rest", "..application.rest..");
}
```

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 9. "Testing microservices: Parte 1". Introduction.
- Libro "Microservices Patterns" (Chris Richardson): Chapter 10. "Testing microservices: Parte 2".
- Libro "Fundamentals of Software Architecture" (Richards & Ford): Y Foundations (Chapter 6).

Criterios de evaluación

- La Práctica 3 se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias se penalizará la actividad con una D como nota.

- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que librar un único **documento** PDF *ApellidosNombre_ISCSDPRAC3.pdf* con la descripción de cómo se ha realizado la práctica (argumentar decisiones e instrucciones de cómo se tienen que ejecutar las pruebas desarrolladas) y **entregar el código de los tests desarrollados** (en un fichero ZIP).

La entrega se tiene que realizar en el espacio de Registro de Evaluación Continuada (RAC) del aula antes de las **23:59 horas del día 6 de junio de 2022**. No se aceptarán entregas fuera de plazo.